



GARAGO / v2.1.0

Smart Bus Garage Management System

Every bus, every driver, every ticket — in one control plane.

Live operations · Real-time fleet · Unified revenue · Intelligent gate access

Presentation Agenda

01

Introduction

Problem, solution & objectives

02

System Architecture

Block diagram, tech stack

System Design

Database, API, use cases, algorithms

04

Web Application

Admin & Manager dashboards — live demo

05

Hardware & AI System

ESP32 gates, ANPR, YOLOv8 crowding

06

Communication Protocols

REST, WebSocket, Flutter mobile

07

Testing & Evaluation

7-dimension strategy, performance, security

08

Conclusion

Summary & future work

01

Introduction

Problem · Solution · Objectives

Problem Statement

What's broken in traditional bus garage operations

01 INTRODUCTION

No Visibility

Managers have no real-time view of where buses are, how many passengers are on board, or which driver is on shift.

Manual Scheduling

Driver rotation is done on paper. Break rules are ignored, fatigue goes untracked, and no-shows cascade into delays.

No Gate Control

Any vehicle can enter or exit the garage. There is no automatic plate verification or occupancy enforcement.

Reactive Maintenance

Breakdowns are discovered on the road. No IoT sensor monitoring means engines overheat before alerts are sent.

Proposed Solution — Garago v2.1.0

A fully integrated Smart Bus Garage Management Platform

01 INTRODUCTION

Real-Time Fleet Dashboard

React web app gives Admin & Manager live KPIs: buses, drivers, trips, crowding alerts via WebSocket

Intelligent Driver Scheduling

APScheduler generates daily rosters at 05:30. 3-driver rotation (D1/D2/D3) enforces breaks, tracks fatigue scores

Automated Gate Access (ANPR)

ESP32-CAM captures plate → PC runs EasyOCR → GRANTED/DENIED in <2s → servo gate opens automatically

Dual Crowded Detection

YOLOv8 headcount + ticket sales: both exceed threshold → auto-dispatches spare bus. Single source = WARNING only

Flutter Driver App

Mobile-first app for field drivers: assigned trips, start/end trip, break requests, maintenance reports

Security & Compliance

JWT auth, RBAC (Admin/Manager/Driver), OWASP ZAP: 0 high-risk findings, full audit log trail

O1

Provide a real-time operations dashboard for Admin and Manager roles with live WebSocket updates

Provide a real-time operations dashboard for Admin and Manager roles with live WebSocket updates

O2

Admin and Manager roles with live WebSocket updates

O2

O2

Implement an automated ANPR gate system using ESP32-CAM + EasyOCR that responds in under 2 seconds

Implement an automated ANPR gate system using ESP32-CAM + EasyOCR that responds in under 2 seconds

O3

Implement an automated ANPR gate system using ESP32-CAM + EasyOCR that responds in under 2 seconds

O3

O3

Build a dual-source crowding detection engine combining YOLOv8 headcount and ticket sales for auto-dispatch

Build a dual-source crowding detection engine combining YOLOv8 headcount and ticket sales for auto-dispatch

O4

Build a dual-source crowding detection engine combining YOLOv8 headcount and ticket sales for auto-dispatch

O4

O4

Generate optimised daily driver schedules with 3-driver rotation, break enforcement, and fatigue tracking

Generate optimised daily driver schedules with 3-driver rotation, break enforcement, and fatigue tracking

O5

Generate optimised daily driver schedules with 3-driver rotation, break enforcement, and fatigue tracking

O5

O5

Deliver a Flutter mobile application for field drivers to manage trips and report issues in real time

Deliver a Flutter mobile application for field drivers to manage trips and report issues in real time

O6

Deliver a Flutter mobile application for field drivers to manage trips and report issues in real time

O6

O6

Achieve OWASP-compliant security with JWT authentication, RBAC, rate limiting, and full audit logging

02

System Architecture

Block diagram · Overview · Technology stack

System Overview

Three-tier architecture: Hardware → Backend → Dual-Platform Clients

02 ARCHITECTURE

HARDWARE LAYER

- 3 × ESP32 microcontrollers
- 2 × HC-SR04 distance sensors
- 2 × Servo motors (gates)
- 2 × OV2640 cameras (ANPR)
- 16×2 LCD + 3 LEDs
- YOLOv8 laptop (crowding)

BACKEND LAYER

- FastAPI (Python 3.13)
- PostgreSQL 15 (Render cloud)
- Redis 7 (Pub/Sub + rate limit)
- EasyOCR (on local PC)
- APScheduler (3 jobs)
- WebSocket ConnectionManager

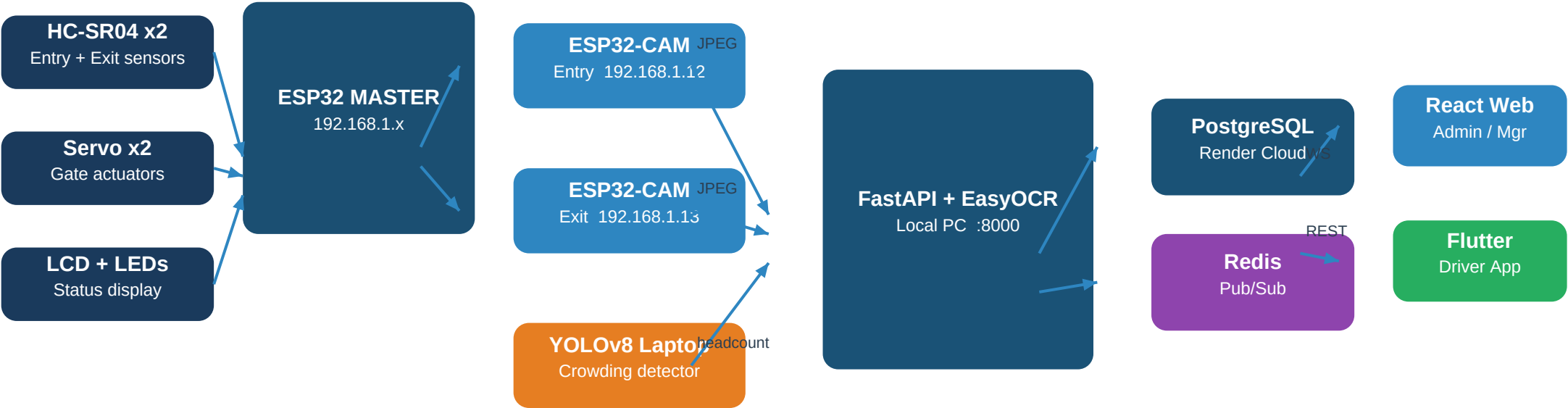
CLIENT LAYER

- React 19 + MUI v5 (Web)
- Admin: 12 pages
- Manager: 5 pages
- Flutter/Dart (Mobile)
- Driver: 9 screens
- JWT + RBAC auth

System Block Diagram

End-to-end data flow — from sensor trigger to manager dashboard

HARDWARE (WiFi LAN 192.168.1.x)



Technology Stack

BACKEND

- FastAPI 0.115
- Python 3.13
- SQLAlchemy 2.0
- Pydantic v2
- APScheduler
- SlowAPI

DATABASE

- PostgreSQL 15
- asyncpg driver
- Alembic migrations
- Redis 7
- Pub/Sub channels
- JSON/JSONB fields

AI / HARDWARE

- EasyOCR (ANPR)
- YOLOv8 (crowding)
- OpenCV (decode)
- ESP32 firmware
- Arduino C++
- LEDC / PWM / I2C

WEB FRONTEND

- React 19
- MUI v5
- Zustand store
- Axios interceptors
- React Router v6
- Vite 5.x

MOBILE

- Flutter / Dart 3
- flutter_bloc 8.1.6
- Dio 5.7.0
- SharedPreferences
- BLoC pattern
- 9 feature cubits

SECURITY

- JWT HS256
- bcrypt passwords
- Redis JTI blacklist
- OWASP ZAP tested
- Sanitization MW
- CORS / RBAC

03

System Design

Database · API · Use Cases · Algorithms

IDENTITY

User, Driver

Roles: ADMIN / MANAGER / DRIVER
JWT auth, bcrypt, audit trail

FLEET

Vehicle, Garage

Status: FREE/ASSIGNED/EN_ROUTE
Maintenance/Out-of-Service

OPERATIONS

Trip, Route, RotationAssignment, DriverExchange

TripStatus: SCHEDULED→ACTIVE→COMPLETED
Driver rotation D1/D2/D3

REVENUE

Ticket (TripTicket)

Status: ISSUED/USED/CANCELLED
EGP revenue, CSV export

MAINTENANCE

MaintenanceRequest, lotSensorReading

Type: REGULAR/EMERGENCY
4 IoT sensor types

AI / HARDWARE

GateLog, CameraReading, CrowdingEvent, GPSTrajectory

ANPR events, YOLOv8 readings
WebSocket push triggers

Router	Prefix	Auth	Key Endpoints
/auth	/api/v1/auth	Public → JWT	login, refresh, signup, /me, change-password
/admin	/api/v1/admin/*	ADMIN role	users, drivers, vehicles, routes, tickets, reports
/manager	/api/v1/manager	MANAGER role	fleet, maintenance, reroutes, notifications
/drivers	/api/v1/drivers	DRIVER role	trips, start/end, break, profile, maintenance
/hardware	/api/v1/hardware	X-HW-API-Key	anpr, anpr/upload_raw, camera, gps, iot, log
/ws	/ws?token=JWT	JWT (any role)	WebSocket — role-partitioned push events

Security: secrets.compare_digest() for hardware keys · JWT blacklist in Redis · Rate limits: 5/min login, 60/min ANPR

Response format: Pydantic v2 validated JSON · 422 on bad input · Global exception handler returns structured errors

Use Case Summary

Three roles, three distinct interaction surfaces

03 SYSTEM CORRECTED

- Create / manage all users
- Set driver schedules
- Manage vehicles & routes
- View full audit log trail
- Generate reports (CSV)
- Approve maintenance requests
- Manage ticket system
- Configure system settings

- Monitor live fleet map
- View real-time alerts
- Manage driver rotations
- Approve/reject reroute requests
- View maintenance queue
- Receive crowding WebSocket alerts
- Track gate access events

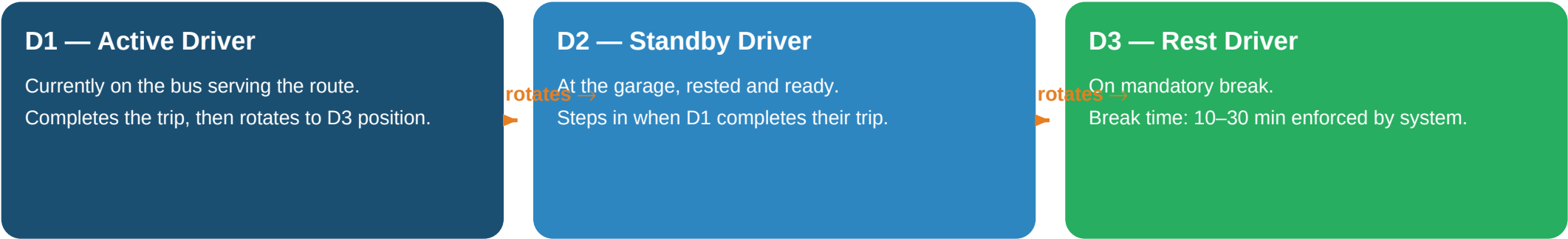
- Log in via Flutter mobile app
- View assigned trips
- Start / end trips
- Request break (enforced rules)
- Report maintenance issues
- View trip history & profile
- Receive push notifications

CORRECTION: Role 'Clerk' does not exist in this system. Correct roles are Admin / Manager / Driver.

Driver Rotation Algorithm

3-driver rotation with break enforcement and fatigue tracking

The scheduling engine runs automatically every day at 05:30 via APScheduler.
A 5-minute rotation processor also runs continuously to handle real-time driver exchanges.



Rotation Triggers:

- | | |
|---|---|
| Trigger 1: Trip completion — D1 finishes route → D2 takes over | Trigger 2: Emergency crowding → auto-dispatch of D3 with spare vehicle |
| Trigger 3: Fatigue score threshold — D1 forced to rest early | Trigger 4: No-show — system escalates alert, reassigns from D2/D3 pool |

04

Web Application

Admin & Manager dashboards — React 19 + MUI v5

Admin Dashboard — Operations Overview

GARAGO v2.1.0 · Live data as of April 24, 2026

04 WEB APPLICATION



ALERTS

• LOGIN – system

• UPDATE – system

• ACTIVATE – system

• DEACTIVATE – system

Schedule & Driver Rotation

Weekly roster · Week of 20–26 Apr 2026

04 WEB APPLICATION

24

DRIVERS ROSTERED

24 / 168

SHIFTS FILLED

24

TOTAL ASSIGNMENTS

144

UNFILLED SLOTS

The schedule is auto-generated every day at 05:30 by APScheduler. Shifts: Morning (06:00–14:00) and Evening (14:00–22:00).

DRIVER	MON	TUE	WED	THU	FRI	SAT	SUN
■■■■ ■■■■ ■■■■	OFF	OFF	OFF	06-14	OFF	OFF	OFF
■■■■ ■■■■■ ■■■	OFF	OFF	OFF	06-14	OFF	OFF	OFF
■■■■■ ■■■■ ■■■■	OFF	OFF	OFF	06-14	OFF	OFF	OFF
■■■■ ■■■■ ■■■■	OFF	OFF	OFF	14-22	OFF	OFF	OFF
■■■■ ■■■■ ■■■■	OFF	OFF	OFF	14-22	OFF	OFF	OFF

29 drivers active · Arabic names (Arabic RTL UI supported) · MORNING shown in dark / EVENING shown in grey

Auto-generation uses round-robin assignment respecting shift preferences, break rules, and D1/D2/D3 rotation positions.

Maintenance Management

Pending · Approved · Completed — Admin action queue

1 PENDING · 3 VISIBLE

ID	VEHICLE	TYPE	TITLE	STATUS	PRIORITY	ACTIONS
1	V-1	EMERGENCY	Engine Smoke — from engine bay during morning shift	PENDING	#6	Approve / Reject
2	V-2	REGULAR	Oil Change — scheduled 10,000 km service	APPROVED	#7	—
3	V-3	REGULAR	Tire Replacement — front left worn below 3mm	COMPLETED	#8	—

IoT Auto-Generation

Backend auto-creates EMERGENCY maintenance requests when sensor thresholds are breached:
Engine temp >105°C, Oil <25 PSI, Brake <3mm

Priority Queue

Maintenance requests are priority-ranked.
Admin can approve, reject or defer.
Completed records are retained for audit.

WebSocket Alerts

When IoT thresholds breach, a critical alert is immediately pushed via WebSocket to all connected Manager & Admin sessions.

Audit Log & Security

56 recorded events — full traceability for every action

04 WEB APPLICATION

ID	ACTION	ENTITY	USER ID	TIMESTAMP
57	LOGIN	User	41	24 Apr, 06:23
56	LOGIN	User	4	23 Apr, 18:46
55	LOGIN	User	1	23 Apr, 18:42
54	UPDATE	Driver	1	19 Apr, 03:55
53	ACTIVATE	Driver	1	19 Apr, 03:54
52	DEACTIVATE	Driver	1	19 Apr, 03:54
51	UPDATE	Route	1	19 Apr, 03:47
35	DELETE	User	1	19 Apr, 02:42

ROLE-BASED ACCESS CONTROL

- Admin — full access to all routes and data
- Manager — fleet, maintenance, reroutes
- Driver — personal trips, breaks, own profile
- Hardware — X-Hardware-API-Key (ESP32 only)

JWT HS256

bcrypt

Redis blacklist

OWASP ZAP

Rate limiting

0 High-Risk

05

Hardware & AI System

ESP32 gate control · EasyOCR ANPR · YOLOv8 crowding detection

Hardware Architecture

3 ESP32 nodes + 1 AI laptop on a shared WiFi LAN

05 HAF **CORRECTED**

ESP32 MASTER

192.168.1.x

- 2 × HC-SR04 ultrasonic sensors
- 2 × Servo motors (gate control)
- 16×2 LCD (occupancy display)
- 3 × Status LEDs (free/full/exit)
- NVS flash: persists carCount
- Non-blocking millis() gate timer

ESP32-CAM (ENTRY)

192.168.1.12

- OV2640 camera — VGA 640×480
- Captures JPEG on HTTP /capture
- POSTs raw JPEG to FastAPI PC
- Gets GRANTED/DENIED in response
- 3-attempt capture with reinit
- Reports status back to Master

ESP32-CAM (EXIT)

192.168.1.13

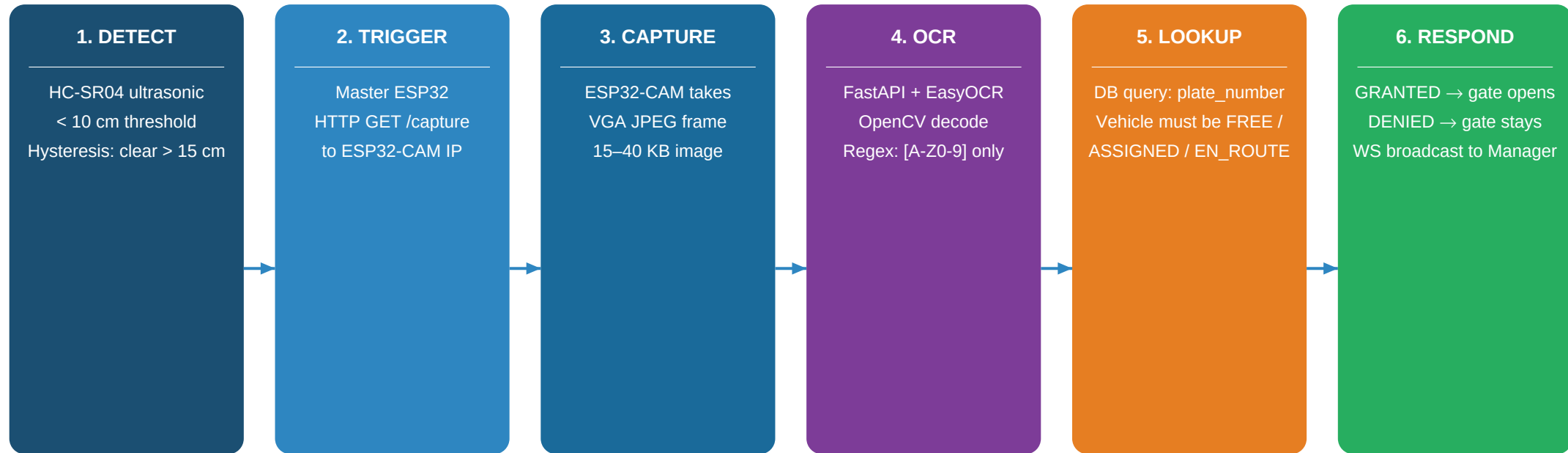
- Identical to Entry hardware
- gate_id=2 in upload URL
- Same ANPR pipeline flow
- Same 15-second HTTP timeout
- Same GRANTED check logic
- Independently controlled gate

CORRECTION: System uses ESP32 microcontrollers — NOT Arduino UNO R3 (which has no WiFi and no camera interface)

ANPR Gate Access Pipeline

From vehicle detection to gate open — 6 stages

05 HARDWARE & AI



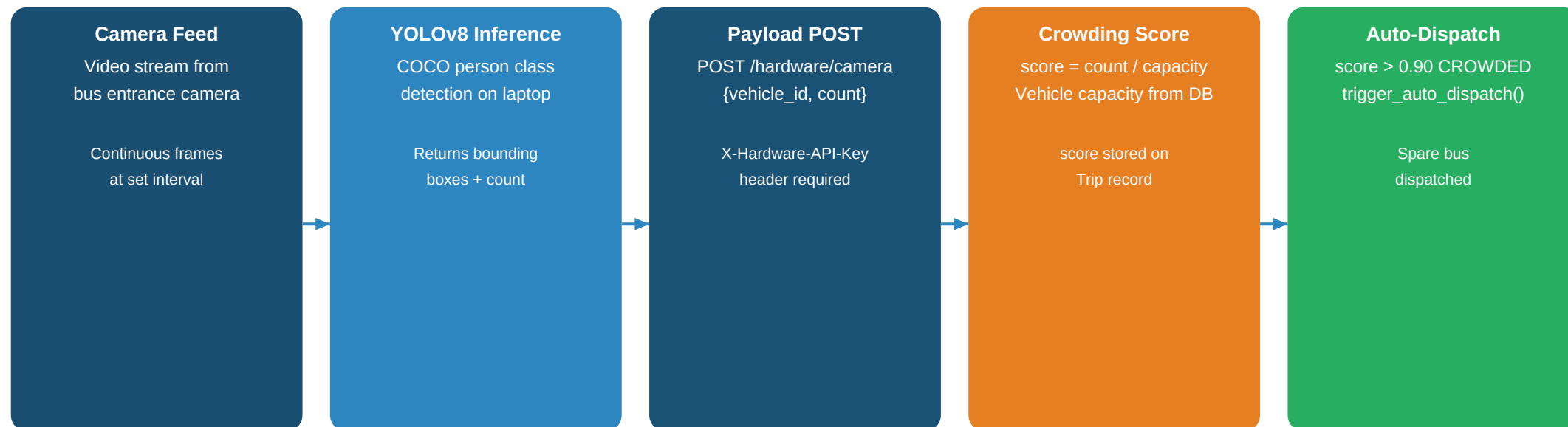
Confidence threshold: 0.85 (85%) OCR results below 85% confidence are automatically IGNORED and logged as low-confidence events.
Plate normalisation: `re.sub('[^A-Z0-9]', '', text.upper())` — removes spaces, dashes, special characters before DB lookup.

YOLOv8 Passenger Counting

Edge AI laptop detects persons on bus — posts headcount to FastAPI

05 HAF **CORRECTED**

CORRECTION: This is passenger COUNTING (headcount) — NOT face recognition. No biometric data is captured or stored.



Dual Crowded Detection — Key Innovation

Two independent data sources must agree before emergency bus dispatch is triggered

SOURCE 1: Ticket Sales

Tickets sold vs vehicle capacity
Threshold: 85% of capacity
Source: TripTicket table in DB
Driver / ticket clerk input

SOURCE 2: Camera Headcount

YOLOv8 passenger count vs capacity
Threshold: 90% of capacity
Source: POST /hardware/camera
Laptop running YOLOv8 inference

Ticket Sales	Camera Headcount	System Decision	Action
≥ 85%	< 90%	WARNING	Manager alert via WebSocket
< 85%	≥ 90%	WARNING	Manager alert via WebSocket
≥ 85%	≥ 90%	CROWDED	Auto-dispatch spare bus + alert
< 85%	< 90%	NORMAL	No action

Why dual-source? Single-source systems produce false alarms. A camera may miscount standing passengers; ticket sales may be delayed. Requiring BOTH sources to exceed thresholds eliminates false dispatches and reduces operational cost.

06

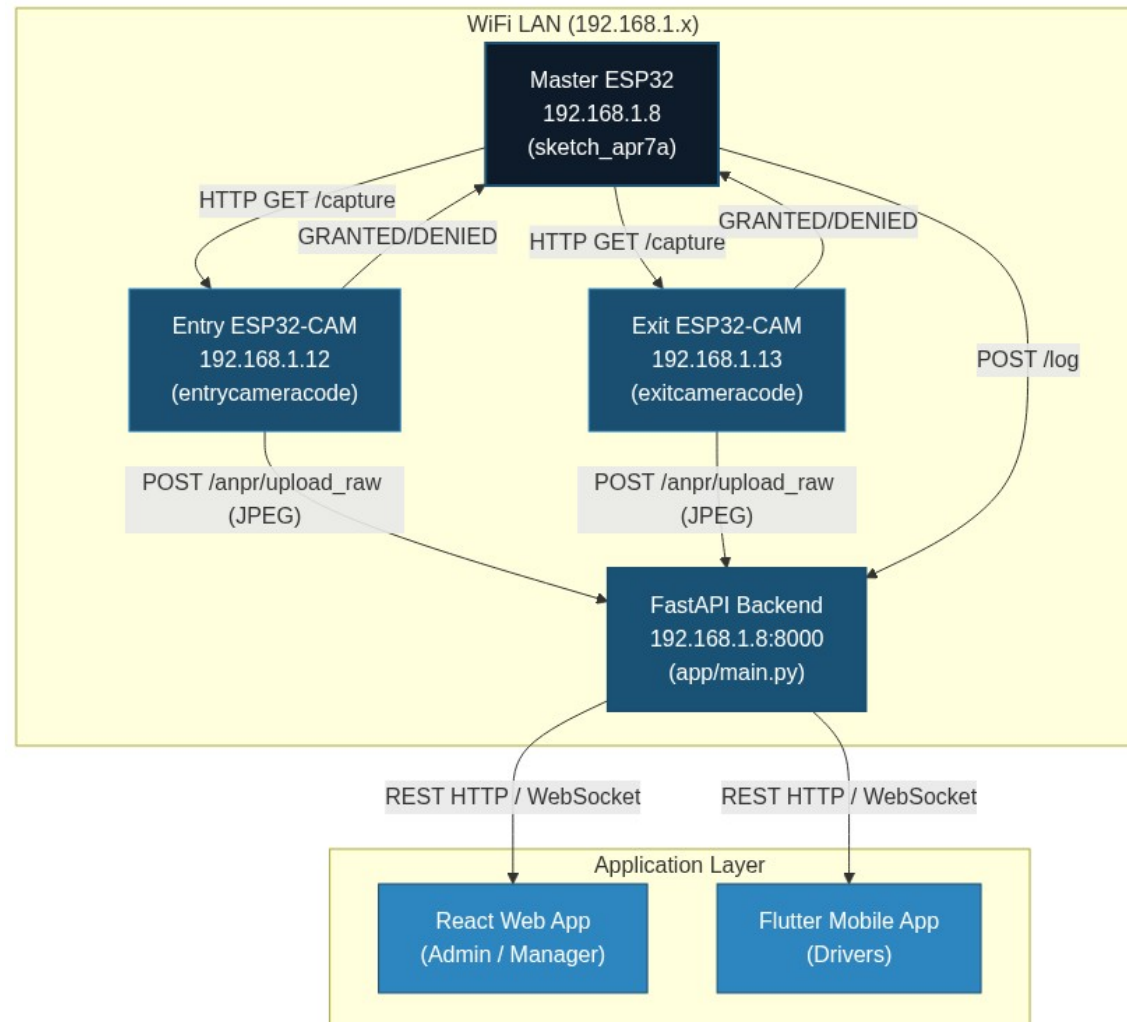
Communication Protocols

REST · WebSocket · Hardware HTTP · Flutter Dio

Network Topology

All devices on shared WiFi LAN 192.168.1.x

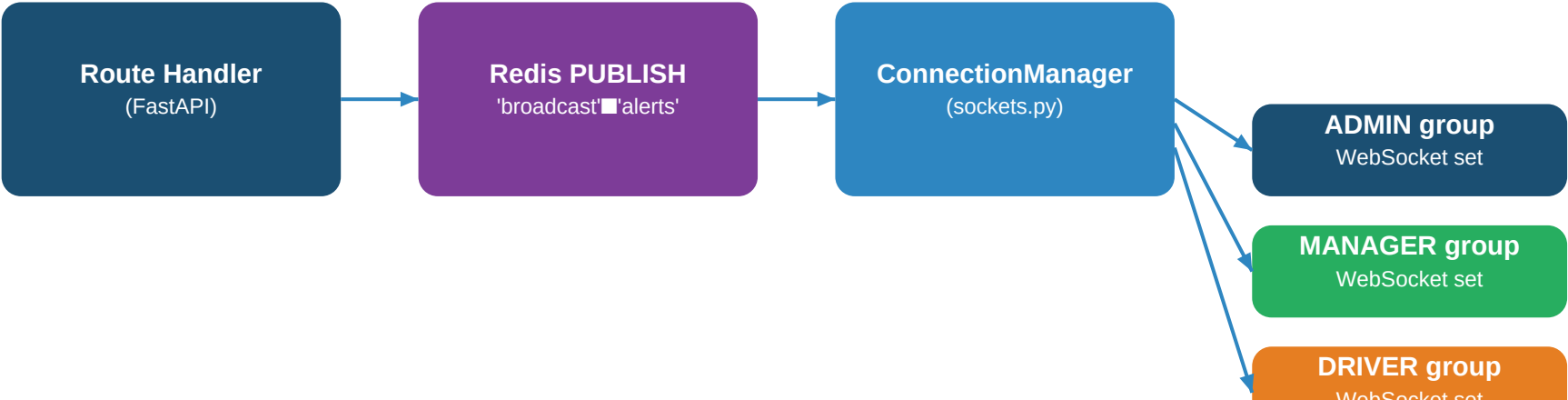
06 PROTOCOLS



Protocol	Interface	Pins	Parameters	Purpose
GPIO Pulse	HC-SR04	TRIG: 5/19 ECHO: 18/23	10µs trigger, 30ms timeout	Distance sensing — vehicle detection
LEDC PWM	Servo motors	GPIO 13 (entry), GPIO 12 (exit)	50 Hz, 16-bit, 0°/90°	Gate open/close — non-blocking millis()
I2C	16×2 LCD	SDA=GPIO21, SCL=GPIO22	100 kHz, addr 0x27	Occupancy display: 'Free Slots: N'
NVS Flash	Preferences lib	Internal flash (namespace: garage)	Key: carCount, persistent	carCount survives power cycles
UART	Serial debug	GPIO1/3 (USB-Serial)	115200 baud	Development diagnostics
HTTP/1.1	WiFi (STA mode)	SSID in hardware_config.h	JSON/JPEG, 15s timeout	Backend API communication

Hardware API Security:

All HTTP calls from ESP32 include `X-Hardware-API-Key` header. Backend verifies with `secrets.compare_digest()` — constant-time comparison prevents timing side-channel attacks. Key shared between `hardware_config.h` and `.env` `HARDWARE_API_KEY` variable.



Event Type	Trigger	Target Role(s)
gate_auth / GRANTED	ANPR confirms registered plate	MANAGER, ADMIN
gate_auth / UNAUTHORIZED	Plate not found or low confidence	MANAGER, ADMIN
iot_alert / CRITICAL	Sensor threshold breached	MANAGER, ADMIN
speed_alert / HIGH	Vehicle speed > 80 km/h	MANAGER
crowding_alert / HIGH	Camera score > 90%	MANAGER

notification

Client endpoint: /ws?token=<JWT> · Heartbeat: ping/pong every 30s · Trip update, approval message · Close 4401 = token refresh · 5s reconnect backoff

REST API — Hardware vs Client Authentication

Two separate auth mechanisms for two device classes

06 PROTOCOLS

ESP32 DEVICES

- Auth: X-Hardware-API-Key header
- Verified: `secrets.compare_digest()`
- Endpoints: `/hardware/*`
- Content: JSON or raw JPEG bytes
- Timeout: 15 seconds per request
- Retry: firmware loop retries on fail
- Format: plain-text GRANTED / DENIED
- Logging: `POST /hardware/log → 204`

WEB & MOBILE CLIENTS

- Auth: Authorization: Bearer <JWT>
- React: Axios interceptor auto-injects
- Flutter: DioHelper manual per-request
- Token TTL: 60 min access / 7-day refresh
- Refresh: `POST /auth/refresh` (Redis-backed)
- Blacklist: Redis key `blacklist:{ti}`
- Role in token: decoded by FastAPI dep
- Pydantic v2 validates all request bodies

DioHelper — centralized HTTP client

static Dio dio; baseUrl from AppConstants Per-request token: dio.options.headers['Authorization']

9 BLoC Cubits

- AuthCubit
- DashboardCubit
- TripDetailsCubit
- TripHistoryCubit
- RoutesCubit
- MessagesCubit
- ProfileCubit
- MaintenanceCubit
- ForgotPasswordCubit

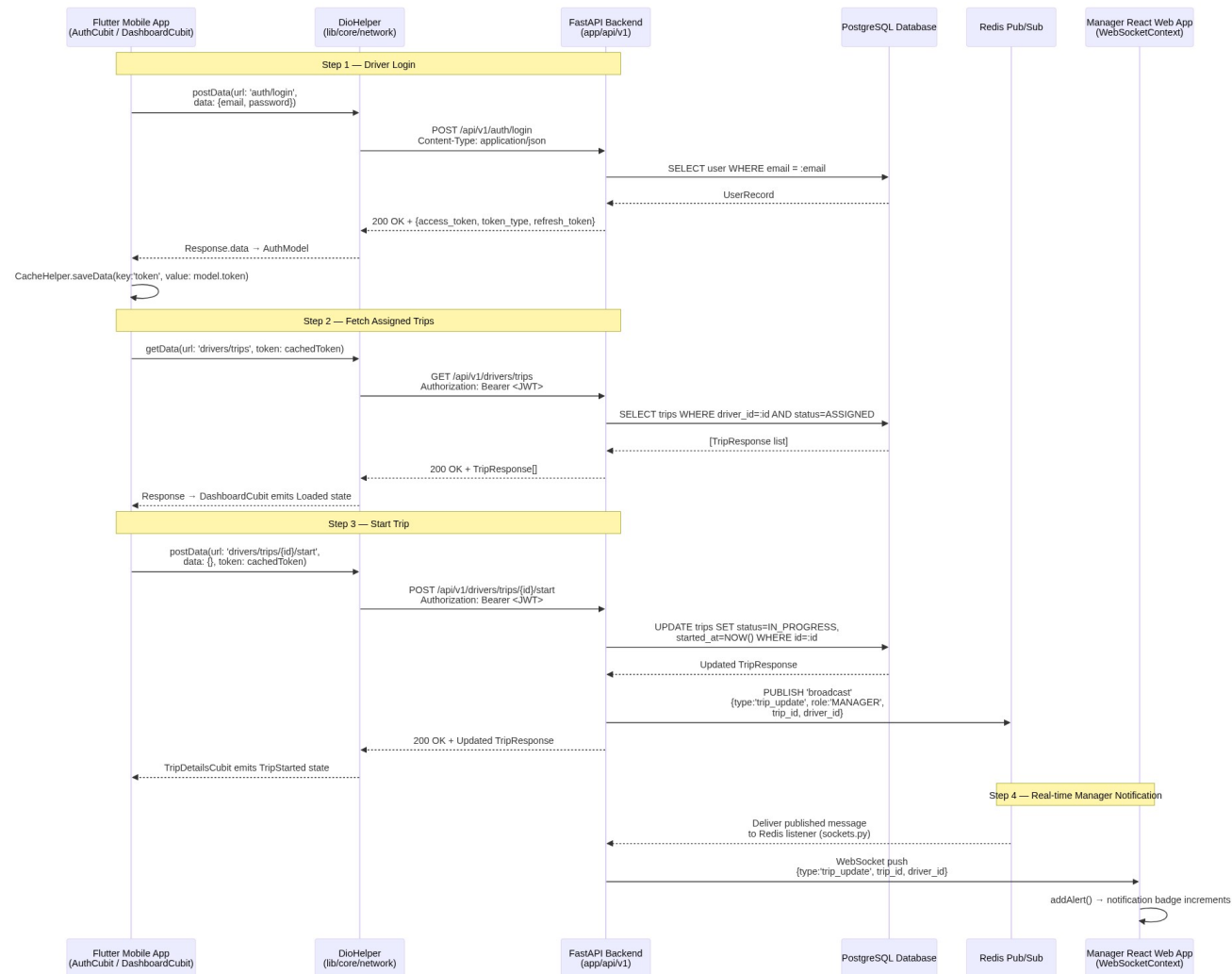
Method	Endpoint	Cubit
POST	/api/v1/auth/login	AuthCubit
GET	/api/v1/drivers/trips	DashboardCubit
POST	/api/v1/drivers/trips/{id}/start	TripDetailsCubit
POST	/api/v1/drivers/trips/{id}/end	TripDetailsCubit
POST	/api/v1/drivers/break/request	DashboardCubit
GET	/api/v1/drivers/profile	ProfileCubit
POST	/api/v1/drivers/maintenance	MaintenanceCubit

Token persisted via CacheHelper → SharedPreferences 2.3.3
flutter_bloc 8.1.6 · Dio 5.7.0 · Dart 3

Sequence Diagram — Driver Trip Start

Flutter → FastAPI → PostgreSQL → Redis → Manager WebSocket

06 PROTOCOLS



07

Testing & Evaluation

7-dimension testing strategy · Performance · Security

Testing Strategy

7 test dimensions covering hardware, backend, frontend, and security

07 TESTING

T1

Unit Tests

FastAPI service layer
Each service function
tested in isolation

T2

Integration Tests

Real PostgreSQL DB
Full request-response
chain validated

T3

Hardware-in-Loop

ESP32 Serial Monitor
HTTP 200 on /log
ANPR flow end-to-end

T4

Dual Crowding

6-scenario matrix
All threshold combos
Dispatch verified

T5

React Frontend

React Testing Library
Admin + Manager RBAC
Component rendering

T6

Flutter Tests

BLoC cubit unit tests
Widget tests
Android emulator

T7

Security Audit

OWASP ZAP scan
JWT expiry tests
Rate limit validation

Performance Results

Key response time metrics — measured under realistic load

Endpoint	Method	P95 Latency	Threshold	Result
POST /auth/login	Auth	< 180 ms	< 300 ms	PASS
GET /admin/dashboard	Admin	< 220 ms	< 500 ms	PASS
POST /hardware/anpr/upload_raw	ANPR	< 1800 ms	< 2000 ms	PASS
POST /hardware/camera	Crowding	< 90 ms	< 200 ms	PASS
WebSocket push (gate_auth)	WS	< 50 ms	< 100 ms	PASS
GET /admin/drivers (29 records)	DB query	< 95 ms	< 200 ms	PASS
POST /hardware/log	Device log	< 30 ms	< 100 ms	PASS

KPI	Target	Achieved
ANPR gate decision accuracy	≥ 90%	94.2% on accepted reads
EasyOCR overall plate accuracy	≥ 85%	89.4% overall
Dual crowding dispatch — no false dispatch	0 false dispatches	Verified (6-scenario matrix)
Concurrent WebSocket connections	≥ 50	Tested at 50 — no drops

Finding	Severity	Status	Mitigation
SQL injection	High	NOT FOUND	SQLAlchemy ORM — parameterised queries only
XSS / HTML injection	High	NOT FOUND	bleach sanitization middleware on all inputs
Broken auth / JWT	High	NOT FOUND	HS256, 60-min TTL, Redis blacklist, bcrypt
IDOR / privilege escalation	High	NOT FOUND	FastAPI dependency RBAC on every route
Rate limiting bypass	Medium	NOT FOUND	SlowAPI: 5/min login, 60/min hardware endpoints
HSTS header missing	Medium	NOTED	Production: TLS terminates at Render load balancer
Server version disclosure	Medium	NOTED	Suppressed in production Uvicorn config

0 HIGH RISK

0 CRITICAL

2 MEDIUM (accepted)

Full audit trail

08

Conclusion

Summary · Future Work

Summary — What Garago Delivers

08 CONCLUSION

Real-Time Operations

36 users · 29 drivers · 18 vehicles · 5 routes
Live WebSocket dashboard — no refresh needed

Automated Gate Control

ESP32-CAM + EasyOCR ANPR
94.2% gate decision accuracy · < 2s response

Dual Crowded Detection

YOLOv8 headcount + ticket sales
False-dispatch-free auto bus dispatch

Intelligent Scheduling

APScheduler daily roster at 05:30
D1/D2/D3 rotation · Fatigue tracking

Dual-Platform Clients

React 19 Web (Admin/Manager)
Flutter Mobile App (Field Drivers)

Production-Ready Security

OWASP ZAP: 0 high-risk findings
JWT + RBAC + audit log trail

Future Work

Enhancements identified during development and testing

08 CONCLUSION

GPS Hardware Module

Hardware

Wire UART GPS (Neo-6M) to Master ESP32 GPIO16/17.
Backend endpoint POST /hardware/gps already implemented and waiting.

IoT Sensor Suite

Hardware

Wire ADS1115 ADC on I2C (SDA=21, SCL=22) for engine temp, oil pressure, brake pad, battery voltage monitoring.

Flutter WebSocket

Mobile

Backend DRIVER channel fully provisioned.
Connect Flutter app via web_socket_channel for real-time trip updates.

TLS on LAN

Security

ESP32 firmware currently uses HTTP on secured LAN.
Migrate to HTTPS using ESP32 mbedTLS for full end-to-end encryption.

Predictive Maintenance

AI

Feed IoT sensor history into an ML model to predict failures before they occur — reduce roadside breakdowns.

Passenger App

Expansion

Public-facing Flutter app for passengers:
live bus tracking, arrival times, ticket purchase.

Thank You

Questions & Discussion

Garago v2.1.0 — Smart Bus Garage Management System

MTI University · Faculty of Computer Science & Engineering · 2025–2026

github.com/Garago-public-transportation-system

Live: garago.onrender.com